

32-bit microcontroller

TIM3 and **ADC** module in motor **FOC** control single
resistor sampling linkage operation instructions

Application Note

Rev1.0 December 2023

Applicable objects

Product series	Product model	Product series	Product model
F series	HC32F030	L series	HC32L130
	HC32F176		HC32L136
	HC32F170		HC32L176
	HC32F072		HC32L170
	HC32F190		HC32L072
	HC32F196		HC32L073
	HC32F420		HC32L190
			HC32L196

Disclaimer

- ★ Xiaohua Semiconductor Co., Ltd. (hereinafter referred to as "XHSC") reserves the right to change, correct, enhance, and modify Xiaohua Semiconductor products and/or this document at any time without prior notice. Users can obtain the latest relevant information before placing an order. XHSC products are sold in accordance with the sales terms and conditions stated in the basic purchase and sales contract.
- ★ Customers should select appropriate XHSC products for your application and design, verify and test your application to ensure that your application meets the corresponding standards and any safety, security or other requirements. Customers shall bear all responsibility for this.
- ★ XHSC hereby confirms that no intellectual property license is granted, either expressly or impliedly.
- ★ Resale of XHSC products, if its terms are different from those specified herein, will invalidate any warranty commitment of XHSC for such products.
- ★ Any graphics or words with "®" or "™" logos are trademarks of XHSC. All other product or service names displayed on XHSC products are the property of their respective owners.
- ★ The information in this notice supersedes and replaces the information in previous versions.

©2023 Xiaohua Semiconductor Co., Ltd. All rights reserved

Table of contents

Applicable objects	2
Disclaimer	3
Table of contents	4
1 Overview	5
2 Implementation of single resistor sampling.....	6
2.1 Timer TIM3 synchronous comparison ADC trigger.....	6
2.1.1 Mode 2/3 complementary PWM output.....	7
2.2 ADC module single resistor sampling application.....	7
2.2.1 ADC channel and trigger source selection.....	7
3 M0+ series single resistor sampling application configuration	9
3.1 Operation overview	9
3.2 Operation process.....	9
3.2.1 Complementary PWM signal generation.....	9
3.2.2 ADC trigger signal generation.....	9
3.2.3 Start TIM3 counting.....	10
3.2.4 Notes.....	10
4 Sample code	11
4.1 Configuration of TIM3 output dead zone PWM and synchronous TIM0 trigger	11
4.2 M0+ series ADC single resistor sampling	14
Revision history.....	15

1 Overview

This application note mainly introduces the linkage operation of Xiaohua Semiconductor's general-purpose MCU control timer module and ADC module when single-resistor sampling is used in motor FOC control.

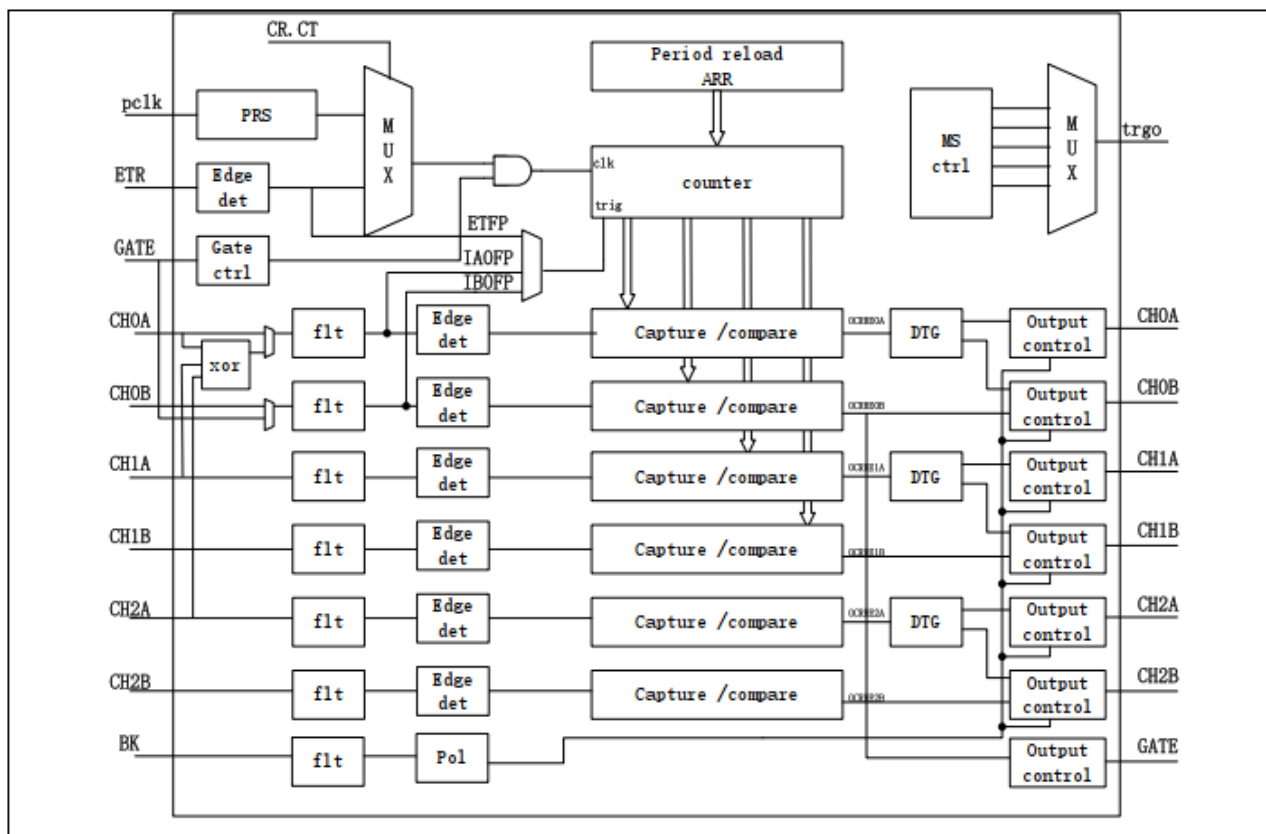
2 Implementation of single resistor sampling

In motor variable frequency control, single resistor sampling will encounter some challenges. When the space vector pulse width modulator (SVPWM) is at the sector boundary and low modulation area of the space vector, there will be two long and one short duty cycles, two short and one long duty cycles, and three almost equal duty cycles. In this case, the effective vector duration is less than the current sampling time. If no phase shift is performed, sampling errors will occur. When the phase shift method is adopted, the register value of the FOC variable frequency control PWM duty cycle changes, but if this register is used for ADC sampling, the noise of the hardware switch at this moment will cause a large error in the sampling, affecting the control accuracy and noise. Therefore, when sampling with a single resistor, sampling delay needs to be considered. At this time, the general MCU chip needs to be able to store the duty cycle value after phase shift, and can generate trigger conditions to trigger ADC synchronous sampling.

Xiaohua's general MCU Cortex M0+ series system has two types of timers that can realize FOC control and synchronous delay trigger ADC sampling functions. One method uses a dedicated comparator to trigger the ADC. For detailed operation, please refer to another AN "Operation Instructions for Single Resistor Sampling Linkage between TIMER4 and ADC Module in Motor FOC Control". The other method uses a synchronous timer to trigger the ADC. The specific implementation of the second method is described in detail below.

2.1 Timer **TIM3** synchronous comparison **ADC** trigger

The general control timer (TIM3) in Xiaohua Semiconductor's general control is a timer composed of 1 counting unit and 6 comparison units. It supports 6 independent PWM outputs or 3 pairs of complementary PWM outputs and can be used as a timer module for three-phase motor control. The timer supports two waveform modes: triangle wave and sawtooth wave, and can generate various PWM waveforms; supports cache function; supports TIM0/1/2/3 synchronization, supports AOS trigger function, supports BRAKE (brake) control, and the Cortex M0 series products are equipped with Timer0/1/2/3 with 1 unit each. The following figure is the basic block diagram of TIM3. It can be seen from the figure that the timer has 3 complementary PWMs with hardware inserted dead zone, which are used to output PWM output in FOC control. In addition, there is a VC comparison output to control the brake function, the external BK port input can control the brake, and the system fail can control the brake function. The software brake function can be realized through CR.BG to control the output port to the set state. In addition, the TRGO output signal can be connected to the ITR signal of other timers for TIMER interconnection, so as to achieve timer synchronization triggering to realize the function that the ADC sampling point needs to be delayed triggered.



2.1.1 Mode 2/3 complementary PWM output

In the dead timer mode (TIMx_DTR.DTEN=01), the internal output signal (in_op*) generated by the comparison match of the value of the general comparison reference register (OCCR*) and the set value of the PWM dead zone control register (PDAR/PDBR) are offset by timing to achieve complementary PWM output in hardware. In this mode, the polarity of the output of the TIM4_<t>_O*H port is the same as in_op*, and the polarity of the output of the TIM4_<t>_O*L port is opposite to in_op*.

2.2 ADC module single resistor sampling application

The 12-bit ADC is an analog-to-digital converter using successive approximation. The analog input channels can be arbitrarily combined into a sequence, and a sequence can be converted by single scan or continuous scan. It supports multiple consecutive conversions of any specified channel and averages the conversion results. The ADC module is also equipped with an analog watchdog function to monitor the conversion results of any specified channel to detect whether they exceed the range set by the user.

2.2.1 ADC channel and trigger source selection

The scan conversion mode is divided into two modes: sequential scan conversion and queue scan conversion. When the two modes work separately, multiple channels can be converted continuously for multiple times; when the two modes work simultaneously, the channels configured for queue scan are converted first.

The sequential scan conversion mode can perform up to 16 consecutive conversions, and the total number of conversions is configured by ADC_SQR2.CNT; all 30 channels can be configured for conversion, and the channels to be converted are configured by ADC_SQRx.CHxMux. This mode can be started by setting the ADC_SqrStart.Start bit or by setting the external trigger of ADC_ExtTrigger0.

The queue-jumping scan conversion mode can perform up to 4 consecutive conversions, and the total number of conversions is configured by ADC_JQR.CNT; all 30 channels can be configured for conversion, and the channels to be converted are configured by ADC_JQR.CHxMux. This mode can be started by setting the ADC_JqrStart.Start bit or by setting the external trigger of ADC_ExtTrigger1. After starting the conversion, the ADC module converts the channels configured in CHxMux~CH0Mux in sequence until the total number of conversions is completed.

In actual applications, the real-time requirements of motor control are high, and current sampling requires real-time feedback, so it is recommended to choose queue-jumping sampling as the configuration of single resistor sampling.

3 M0+ series single resistor sampling application configuration

3.1 Operation overview

The operation flow of Xiaohua Semiconductor's general control timer module in the motor FOC single resistor sampling control is as follows:

- 1) Generate 3 sets of complementary dead-zone and cached PWM signals and write them into the comparison configuration register to drive the three-phase motor.
- 2) Enable the CCRx comparison trigger function in the synchronous TIME0/1/2 interconnection function, select immediate effect, generate an ADC trigger signal at a specific time, and trigger the ADC to sample the current signal.
- 3) Enable the ADC conversion completion trigger DMA function, configure the DMA source address and destination address for ADC output transfer or trigger the interrupt enable to read data in the interrupt.
- 4) Start the ADC, start TIM3 and synchronous TIME0/1/2 counting, start output and sampling (TIM0 is used as the synchronous counting trigger timer in this article).

3.2 Operation process

3.2.1 Complementary PWM signal generation

- 1) Configure peripheral clock, counter count value, count mode, IO status
In FOC control, the carrier in most application scenarios selects triangular wave mode, generates a triangular carrier with the target cycle frequency, and generates a count overflow interrupt or a count underflow interrupt (taking the count overflow interrupt as an example).
- 2) Configure comparison output level, event update mode, and cache enable
Set the port output selection corresponding to the comparison output, the comparison output signal source, enable the event update mode, and the comparison output mode.
- 3) PWM output process
Set the PWM output mode (complementary dead zone mode), whether the PWM output polarity is flipped, the GPIO initial output level, the width of the dead zone, and other PWM output information.
- 4) Configure synchronous TIM0 according to 1/2 steps, enable TIM0 as a single-point trigger comparison mode, and no PWM output.
- 5) Configure TIM3 as the master output mode in synchronous startup, TIM0 as the slave output mode, and select TIM3 for synchronous trigger information.

3.2.2 ADC trigger signal generation

When the TIM3 module is running, according to the characteristics of the single resistor sampling time point, the comparison trigger sampling cannot be generated immediately at the comparison point of the CCR (the sampling has errors), and TIM3 has no dedicated comparator and trigger event, so the synchronous TIM0 is used, and the comparison value register of TIM0 is used to load the comparison value after the TIM3 comparison duty cycle delay to trigger the single resistor sampling. In the set counting direction, it is necessary to match the ADC sampling twice at any value of [0-cycle value], and the interval between the two duty cycle values issued cannot be less than the current sampling time. Therefore, two comparison registers are needed as the trigger source for triggering ADC events. Configure the peripheral ADC and TIM clock, and enable the ADC queue sampling mode.

- 1) Configure ADC channel and sampling time;
- 2) Configure the trigger source of ADC to be TIM0;
- 3) Enable DMA or interrupt to receive the sampled value.

3.2.3 Start **TIM3** counting

Start TIM3 counting and synchronously start TIM0 to start counting.

```
en_result_t Tim3_M23_Run(void)
{
    en_result_t enResult = Ok;
    MOP_TIM3_MODE23->M23CR.f.CTEN = TRUE;
    return enResult;
}
```

3.2.4 Notes

- 1) It is recommended to enable the cache function to ensure the timing in FOC control and prevent unexpected problems.
- 2) ADC sequential scanning and queue-interrupting scanning cannot be performed at the same time and there should be no conflict.
- 3) The rising edge and falling edge triggering methods can be changed according to the different sampling current reconstruction methods.
- 4) The polarity of the PWM output can be matched according to the polarity of the drive, and the trigger point can be changed synchronously.
- 5) The sampling time needs to be set according to the sampling clock and external load.
- 6) For the improvement of ADC sampling accuracy, please refer to AN "AN_How to improve ADC sampling accuracy_Rev1.0".
- 7) In the two-way PWM comparison trigger sampling (sample) mode of TIM3 synchronizing TIM0, it is necessary to read data by interrupt mode, and software counting is required to determine the sampling order.
- 8) TIM3 can also be used to synchronize the two timers in TIM0/1/2, and use different TIM comparisons to trigger ADC. At this time, data can be read by DMA or interrupt mode, without software determining the sampling order.

4 Sample code

The following is an introduction to the configuration of related peripherals in motor FOC control based on TIM3 and ADC modules in this AN. The example uses HC32F170 chip.

4.1 Configuration of **TIM3** output dead zone **PWM** and synchronous **TIM0** trigger

```
void App_Timer3Cfg(uint16_t u16Period, uint16_t u16DeadTimeCnt, uint8_t u8PeakIsrMaskCnt)
{
    uint16_t          u16CntValue;
    uint16_t          u16CHxACmpare;
    stc_tim3_mode23_cfg_t    stcTim3BaseCfg;
    stc_tim3_m23_compare_cfg_t    stcTim3PortCmpCfg;
    stc_tim3_m23_adc_trig_cfg_t    stcTim3TrigAdc;
    stc_tim3_m23_dt_cfg_t    stcTim3DeadTimeCfg;
    stc_tim3_m23_master_slave_cfg_t    stcTim3MasterSlaveCfg;

    /*Structure initialization clearing*/

    DDL_ZERO_STRUCT(stcTim3BaseCfg);
    DDL_ZERO_STRUCT(stcTim3PortCmpCfg);
    DDL_ZERO_STRUCT(stcTim3TrigAdc);
    DDL_ZERO_STRUCT(stcTim3DeadTimeCfg);

    /*Timer3 peripheral clock enable*/

    Sysctrl_SetPeripheralGate(SysctrlPeripheralTim3, TRUE);

    /*TIM3 mode 23 function initialization*/

    stcTim3BaseCfg.enWorkMode    = Tim3WorkMode3;
    stcTim3BaseCfg.enCT          = Tim3Timer;
    stcTim3BaseCfg.enPRS         = Tim3PCLKDiv1;
    stcTim3BaseCfg.enPWMTypeSel  = Tim3ComplementaryPWM;
    stcTim3BaseCfg.enPWM2sSel    = Tim3DoublePointCmp;
    stcTim3BaseCfg.bOneShot      = FALSE;
    stcTim3BaseCfg.bURSSel       = FALSE;
    Tim3_Mode23_Init(&stcTim3BaseCfg);
    Tim3_M23_ARRSet(u16Period, TRUE);
    u16CHxACmpare = 0;
    Tim3_M23_CCR_Set(Tim3CCR0A, u16CHxACmpare);
    Tim3_M23_CCR_Set(Tim3CCR1A, u16CHxACmpare);
    Tim3_M23_CCR_Set(Tim3CCR2A, u16CHxACmpare);

    /*Comparison output port configuration*/

    stcTim3PortCmpCfg.enCHxACmpCtrl    = Tim3PWMMode2;
    stcTim3PortCmpCfg.enCHxAPolarity    = Tim3PortPositive;
    stcTim3PortCmpCfg.bCHxACmpBufEn    = TRUE;
```

```

    stcTim3PortCmpCfg.enCHxACmplntSel = Tim3CmplntNone;
    stcTim3PortCmpCfg.enCHxBCmpCtrl   = Tim3PWMMMode2;
    stcTim3PortCmpCfg.enCHxBPolarity  = Tim3PortPositive;
    stcTim3PortCmpCfg.bCHxBCmpBufEn = TRUE;
    stcTim3PortCmpCfg.enCHxBCmplntSel = Tim3CmplntNone;
    Tim3_M23_PortOutput_Cfg(Tim3CH0, &stcTim3PortCmpCfg);
    Tim3_M23_PortOutput_Cfg(Tim3CH1, &stcTim3PortCmpCfg);
    Tim3_M23_PortOutput_Cfg(Tim3CH2, &stcTim3PortCmpCfg);

    /*TIM synchronous master-slave mode configuration*/

    stcTim3MasterSlaveCfg.enMasterSlaveSel = Tim3MasterMode;
    stcTim3MasterSlaveCfg.enMasterSrc = Tim3MasterCTEN;
    Tim3_M23_MasterSlave_Set(&stcTim3MasterSlaveCfg);

    /*TIM dead zone and event update cycle configuration*/

    stcTim3DeadTimeCfg.bEnDeadTime      = TRUE;
    stcTim3DeadTimeCfg.u8DeadTimeValue  = u16DeadTimeCnt;
    Tim3_M23_DT_Cfg(&stcTim3DeadTimeCfg);
    Tim3_M23_SetValidPeriod(u8PeakIsrMaskCnt);
    u16CntValue = 0;
    Tim3_M23_Cnt16Set(u16CntValue);
    Tim3_ClearAllIntFlag();
    Tim3_Mode23_EnableIrq(Tim3UevIrq);
    EnableNvic(TIM3_IRQn, IrqLevel1, TRUE);
}

void App_Timer0Cfg(uint16_t u16Period,  uint8_t u8PeakIsrMaskCnt)
{
    uint16_t          u16CntValue;
    stc_bt_mode23_cfg_t    stcBtBaseCfg;
    stc_bt_m23_compare_cfg_t    stcBtPortCmpCfg;
    stc_bt_m23_adc_trig_cfg_t    stcBtTrigAdc;
    stc_bt_m23_dt_cfg_t          stcBtDeadTimeCfg;
    stc_bt_m23_master_slave_cfg_t    stcBtMasterSlaveCfg;
    DDL_ZERO_STRUCT(stcBtBaseCfg);
    DDL_ZERO_STRUCT(stcBtPortCmpCfg);
    DDL_ZERO_STRUCT(stcBtTrigAdc);
    DDL_ZERO_STRUCT(stcBtDeadTimeCfg);
    DDL_ZERO_STRUCT(stcBtMasterSlaveCfg);
    Sysctrl_SetPeripheralGate(SysctrlPeripheralBaseTim, TRUE);

    /*TIM0 mode 23 function initialization*/

    stcBtBaseCfg.enWorkMode      = BtWorkMode3;
    stcBtBaseCfg.enCT            = BtTimer;
    stcBtBaseCfg.enPRS           = BtPCLKDiv1;

```

```

//stcBtBaseCfg.enCntDir    = BtCntUp;
stcBtBaseCfg.enPWMTypeSel  = BtComplementaryPWM;

stcBtBaseCfg.enPWM2sSel    = BtSinglePointCmp;           //Single point comparison function

stcBtBaseCfg.bOneShot      = FALSE;
stcBtBaseCfg.bURSSel      = FALSE;
Bt_Mode23_Init(TIM0, &stcBtBaseCfg);
Bt_M23_ARRSet(TIM0, u16Period, TRUE);
Bt_M23_CCR_Set(TIM0, BtCCR0A, 200);
Bt_M23_CCR_Set(TIM0, BtCCR0B, 1000);
stcBtPortCmpCfg.enCH0ACmpCtrl  = BtPWMMode2;
stcBtPortCmpCfg.enCH0APolarity = BtPortPositive;
stcBtPortCmpCfg.bCh0ACmpBufEn  = TRUE;
stcBtPortCmpCfg.enCh0ACmplntSel = BtCmplntFall;
stcBtPortCmpCfg.enCH0BCmpCtrl  = BtPWMMode2;
stcBtPortCmpCfg.enCH0BPolarity = BtPortPositive;
stcBtPortCmpCfg.bCH0BCmpBufEn  = TRUE;
stcBtPortCmpCfg.enCH0BCmplntSel = BtCmplntFall;
Bt_M23_PortOutput_Cfg(TIM0, &stcBtPortCmpCfg);

/*TIM0 synchronous slave mode and trigger ADC configuration*/

stcBtTrigAdc.bEnCH0ACmpTrigADC = FALSE;
stcBtTrigAdc.bEnCH0BCmpTrigADC = TRUE;
stcBtTrigAdc.bEnUevTrigADC = FALSE;
stcBtTrigAdc.bEnTrigADC      = TRUE;
Bt_M23_TrigADC_Cfg(TIM0, &stcBtTrigAdc);
stcBtMasterSlaveCfg.enMasterSlaveSel = BtSlaveMode;
stcBtMasterSlaveCfg.enSlaveModeSel = BtSlaveTrigMode;
stcBtMasterSlaveCfg.enTsSel = BtTs4TIM3TRGO;
Bt_M23_MasterSlave_Set(TIM0, &stcBtMasterSlaveCfg);
Bt_M23_SetValidPeriod(TIM0, u8PeakIsrMaskCnt);
u16CntValue = 0;
Bt_M23_Cnt16Set(TIM0, u16CntValue);
Bt_ClearAllIntFlag(TIM0);
}

```

4.2 M0+ series ADC single resistor sampling

```
void App_AdcCfg(void)
{
    stc_adc_cfg_t          stcAdcCfg;
    stc_adc_jqr_cfg_t      stcAdcJqrCfg;
    stc_adc_sqr_cfg_t      stcAdcSqrCfg;
    DDL_ZERO_STRUCT(stcAdcCfg);
    DDL_ZERO_STRUCT(stcAdcJqrCfg);
    Sysctrl_SetPeripheralGate(SysctrlPeripheralAdcBgr, TRUE);

    Gpio_SetAnalogMode(GpioPortA, GpioPin0);          //PA00 Configure as an analog channel

    /*ADC module initialization configuration*/

    Adc_Enable();

    M0P_BGR->CR_f.BGR_EN = 0x1u; //BGR must be enabled

    M0P_BGR->CR_f.TS_EN = 0x0u;
    delay100us(1);

    stcAdcCfg.enAdcMode          = AdcScanMode;          //Continuous sampling mode

    stcAdcCfg.enAdcClkDiv = AdcMskClkDiv2; //Adc working clock PCLK/2

    stcAdcCfg.enAdcSampCycleSel = AdcMskSampCycle8Clk; //Sampling clock 8 cycles

    stcAdcCfg.enAdcRefVolSel = AdcMskRefVolSelAVDD; //Internal AVDD

    stcAdcCfg.enAdcOpBuf          = AdcMskBufDisable;    //Internal voltage follower off

    stcAdcCfg.enInRef              = AdcMskInRefDisable; //Internal reference voltage Disable

    Adc_Init(&stcAdcCfg);

    /*Configure queue scan conversion */

    Adc_CfgJqrChannel(AdcQRCH0MUX, AdcExInputCH0);

    stcAdcJqrCfg.bJqrDmaTrig = FALSE; //Disable JQR conversion trigger DMA

    stcAdcJqrCfg.u8JqrCnt = 1;          //Conversion start channel (3-1 has been calculated in the library function)

    Adc_JqrModeCfg(&stcAdcJqrCfg);      //Configure queue-jumping scan conversion mode

    Adc_JqrExtTrigCfg(AdcMskTrigTimer0, TRUE);          //Timer0 triggers queue-jumping scan conversion

    Adc_EnableIrq();                                  //Enable Adc interrupt

    EnableNvic(ADC_DAC_IRQn, IrqLevel0, TRUE);          //Adc interrupt
}
```

Revision history

Version	Date	Revision Content
Rev1.0	2023/12/21	Initial version released.